arXiv:2603.16021v2 [cs.AI] 18 Mar 2026

Interpretable Context Methodology: Folder Structure as Agent Architecture

Jake Van Clief, David McDermott

theceo@eduba.io
Eduba, University of Edinburgh Palm CoastFlorida USA

Abstract.

Current approaches to AI agent orchestration typically involve building multi-agent frameworks that manage context passing, memory, error handling, and step coordination through code. These frameworks work well for complex, concurrent systems. But for sequential workflows where a human reviews output at each step, they introduce engineering overhead that the problem does not require. This paper presents Interpretable Context Methodology (ICM), a method that replaces framework-level orchestration with filesystem structure. Numbered folders represent stages. Plain markdown files carry the prompts and context that tell a single AI agent what role to play at each step. Local scripts handle the mechanical work that does not need AI at all. The result is a system where one agent, reading the right files at the right moment, does the work that would otherwise require a multi-agent framework. This approach applies ideas from Unix pipeline design, modular decomposition, multi-pass compilation, and literate programming to the specific problem of structuring context for AI agents. The protocol is open source under the MIT license.[1]

[1] https://github.com/RinDig/Interpretable-Context-Methodology-ICM-

context engineering, human-AI interaction, AI agent orchestration, filesystem architecture, human-in-the-loop, mixed-initiative systems, workflow automation

†

†ccs: Human-centered computing Interactive systems and tools

†

†ccs: Human-centered computing HCI design and evaluation methods

†

†ccs: Computing methodologies Artificial intelligence

†

†ccs: Software and its engineering Software design engineering

1. Introduction

There are genuinely good agentic frameworks available today. CrewAI, LangChain, AutoGen, and others handle multi-step orchestration, memory management, tool use, and error recovery. They work. But they work within their own structures, and adjusting those structures requires development work. Changing the order of steps, swapping a prompt, adding or removing a stage, skipping something that is not relevant today: these actions typically mean editing code, understanding abstractions, and redeploying. For practitioners whose workflows are sequential and need human review at each step, the control surface can be much simpler.

This paper describes Interpretable Context Methodology (ICM), a method for orchestrating AI agent workflows using folder structure, markdown files, and local scripts. The central observation is straightforward: if the prompts and context for each stage of a workflow already exist as files in a well-organized folder hierarchy, you do not need a coordination framework to manage multiple specialized agents. You need one orchestrating agent that reads the right files at the right moment. The folder structure tells it what to do at each step, and if the agent delegates sub-tasks, the same folder structure determines what context those sub-agents receive. Local Python scripts handle the parts that do not need AI: fetching data, moving files, formatting output, sending emails.

This is going backward before going forward. The principles that made Unix pipelines effective in the 1970s[2]

[2]Programs that do one thing. Output of one becomes input of another. Plain text as universal interface. These ideas are over fifty years old and they hold up.

and multi-pass compilers tractable in the 1980s apply directly to AI agent orchestration in the 2020s. ICM applies those principles to the specific challenge of structuring context for language models.

The central question this paper examines is how structuring the context delivery mechanism as a filesystem hierarchy affects practitioners' ability to control, inspect, and edit AI agent behavior across multi-step workflows, and what this structure means for the quality of the model's output at each stage.

The paper is organized as follows. Section 2 traces the relevant background across software engineering, context engineering, and human oversight research. Section 3 describes the protocol itself. Section 4 walks through working implementations and reports on early practitioner experience.

Section 5 discusses where this approach fits and where it does not, including implications for the design of interactive intelligent systems more broadly. Section 6 explores future directions, drawing on the structural parallels between ICM and multi-pass compilation to propose semantic debugging and source-level traceability for AI workflows.

Table 1. Comparison of control surfaces for sequential, human-reviewed workflows. The first six rows show dimensions where ICM's filesystem approach simplifies common operations. The last four rows show dimensions where framework-based approaches provide capabilities that ICM lacks or handles less well.

Dimension	Framework approach	ICM approach
Change stage order	Edit orchestration code, redeploy	Rename or reorder folders
Modify a prompt	Edit agent configuration in code	Edit a markdown file
Add or remove a stage	Write new agent class, update orchestrator	Add or delete a folder
Inspect intermediate state	Add logging, build dashboard	Open the folder, read the files
Hand off to another person	Document environment, dependencies, setup	Copy the folder
Who can make changes	Developer	Anyone with a text editor
Error recovery mid-pipeline	Built-in retry, fallback, exception handling	Manual re-run of failed stage
Conditional branching	Programmatic routing based on agent output	Human decides between stages
Concurrent execution	Native parallel agent coordination	Sequential by design
External service integration	Programmatic API calls, auth management	Local scripts or MCP connections

2. Background and Related Work

2.1. Composability and the Unix Tradition

In 1978, Doug McIlroy articulated the principles that would define Unix's design philosophy: make each program do one thing well, expect the output of every program to become the input to another, and use text streams as the universal interface between programs (mcilroy1978,). These principles were not theoretical. They were engineering decisions driven by constraints. The PDP-11 machines that ran early Unix had limited memory. Programs had to be small. The way to build powerful systems from small programs was to connect them through a common interface (ritchie1974,).

Kernighan and Pike later argued that the power of a Unix system comes more from the relationships among programs than from the programs themselves (kernighan1984,). Eric Raymond codified this

into explicit design rules: the Rule of Modularity (write simple parts connected by clean interfaces), the Rule of Transparency (design for visibility to make inspection and debugging easier), and the Rule of Composition (design programs to be connected to other programs) (raymond2003,).

These principles were formalized in software architecture as the "pipe-and-filter" pattern by Shaw and Garlan (shaw1996,): a system of independent components, each reading from inputs and writing to outputs, connected by data streams. The pattern's strength is that any component can be replaced, inspected, or tested independently.

A related lineage runs through build systems. Stuart Feldman's Make (1979) established that workflows could be defined as dependency graphs between files using declarative specifications (feldman1979,). The key insight: files are both the artifacts of work and the coordination mechanism between stages. You do not need a separate orchestration layer when the filesystem tracks what has been produced and what depends on it. Multi-pass compilers work on the same principle: source code transforms through a sequence of intermediate representations, each pass reading the output of the previous pass, with well-defined interfaces between them (aho2006,).

David Parnas argued in 1972 that systems should be decomposed based on what each module hides from the rest of the system, yielding components that can be modified independently (parnas1972,). Edsger Dijkstra coined the term "separation of concerns" to describe the discipline of addressing one thing at a time as the only available technique for effective ordering of one's thoughts (dijkstra1974,).

These ideas appear across decades and contexts because they describe something real about how systems stay manageable as they grow. They are relevant here because the problem of orchestrating AI agents through multi-step workflows is, at its core, a problem of modular decomposition, clean interfaces, and readable intermediate representations.

2.2. Context Engineering and Agentic AI

The practitioner community has increasingly adopted the term "context engineering" to describe what building production AI systems actually involves. Andrej Karpathy gave the term its clearest articulation in June 2025, arguing that "prompt engineering" understates the work (karpathy2025,). The distinction is useful. Prompt engineering suggests crafting a single instruction. Context engineering describes the broader discipline of filling the context window with the right information: instructions, retrieved knowledge, memory, tool descriptions, and prior outputs, all structured so the model can use them effectively. This paper uses the term in that sense.

Lance Martin at LangChain formalized this into a taxonomy of strategies: write (author instructions), select (choose relevant context), compress (reduce token waste), and isolate (keep unrelated context separate) (martin2025,). Simon Willison argued that the entire information environment, including

previous model responses and system state, is part of the context that needs engineering (willison2025,).

The current generation of agentic frameworks, LangChain (chase2022,), AutoGen (wu2023autogen,), CrewAI, and others, handle context engineering through code-level abstractions. They define agents as objects, conversations as message arrays, and orchestration as programmatic control flow. This works well for systems that need dynamic multi-agent collaboration, concurrent execution, or complex branching logic.

But for sequential workflows, these frameworks solve a coordination problem that may not need to exist. If Agent A's job is to research, Agent B's job is to filter, and Agent C's job is to write, the framework's role is to pass the right context to the right agent at the right time. That coordination can also be achieved by putting the right files in the right folders. The orchestrating agent reads different instructions at each stage. If it delegates sub-tasks to smaller models (as current agent-team architectures allow), the folder structure provides the context for those delegations too. The coordination logic lives in the filesystem, not in application code.

This matters because of how language models handle context. Liu et al. demonstrated that LLMs perform significantly worse when relevant information is buried in the middle of long contexts (liu2024,). The more irrelevant material in the context window, the worse the model performs on the material that matters. Jiang et al. showed that prompt compression can achieve up to 20x token reduction with minimal performance loss (jiang2023,), but a simpler approach is to avoid loading irrelevant context in the first place. Stage-specific context loading, where each stage only sees the files it needs, prevents the problem rather than treating it after the fact.

It is worth distinguishing ICM from Anthropic's Model Context Protocol (MCP) (anthropic2024mcp,). MCP standardizes how models access external tools and data sources, solving the integration problem between AI systems and the services they need to call. ICM addresses a different layer: how to structure and deliver context to an agent across a multi-stage workflow. The two are complementary. An ICM stage might use MCP connections to access external services, while the stage's folder structure determines what context the agent receives when doing so. This separation matters for efficiency as well. Jones and Kelly at Anthropic observed that loading all tool definitions upfront into the context window slows agents and increases costs (jones2025,). ICM's stage-based architecture avoids this by scoping tool definitions to individual stages, loading only the tools relevant to the current step.

2.3. Human Oversight and Observability

The question of how humans should relate to automated systems has been studied for decades, and the findings are remarkably consistent.

Fails and Olsen introduced the interactive machine learning paradigm in 2003: rapid cycles of system output, human feedback, and correction (fails2003,). Amershi et al. argued that interactive ML

must involve users at all stages, from training through evaluation, with interfaces that support steering and correction (amershi2014,). Dudley and Kristensson's review of interface design for interactive ML emphasized that transparent, inspectable representations are essential for effective human-AI collaboration (dudley2018,).

Eric Horvitz's work on mixed-initiative systems established principles for coupling automated services with human control (horvitz1999,). The key insight: systems should let users invoke, adjust, and terminate automated processes at natural breakpoints. This requires that the system's state be visible and its actions be reversible.

Parasuraman and Riley identified the failure modes that emerge when this goes wrong (parasuraman1997,). When automated outputs are opaque, people either trust them blindly (misuse) or stop using them entirely (disuse). Both failures stem from the same cause: the human cannot see what happened between input and output. Lee and See's work on trust calibration reinforced this: appropriate trust requires that system behavior be observable (lee2004,). Parasuraman, Sheridan, and Wickens proposed a taxonomy of automation levels, noting that the right level of automation varies by task and that systems should support different levels at different stages (parasuraman2000,).

Ben Shneiderman synthesized these threads into the Human-Centered AI framework, arguing that systems can achieve both high human control and high automation simultaneously (shneiderman2020,). The two are not in tension. They reinforce each other when the system is designed to be comprehensible, predictable, and controllable (shneiderman2022,).

Cynthia Rudin made the most forceful version of this argument: stop building opaque systems and then trying to explain them after the fact. Build systems that are inherently interpretable (rudin2019,). This applies at the workflow level as much as at the model level. A production pipeline where every intermediate output is a readable file is inherently interpretable. There is nothing to explain because nothing was hidden.

This is also becoming a regulatory concern. The EU AI Act requires human oversight of high-risk AI systems, distinguishing between human-in-the-loop, human-on-the-loop, and human-in-command approaches (enqvist2023,). Novelli et al. argue that effective oversight requires institutional design, not just technical capability (novelli2024,). Systems with staged review points, audit trails, and defined intervention surfaces have a practical advantage as these requirements take effect.

3. Interpretable Context Methodology

3.1. Design Principles

ICM is built on five principles, each borrowed from established practice.

One stage, one job. Each stage in a workspace handles a single step of the workflow and writes its output to its own folder. This follows McIlroy's Unix principle and Parnas's information-hiding criterion (mcilroy1978,; parnas1972,). A stage that fetches data does not also filter it. A stage that filters does not also format the final output. Each stage reads a defined input, transforms it, and writes a defined output, the same structure that governs individual passes in a multi-pass compiler.

Plain text as the interface. Stages communicate through markdown and JSON files. No binary formats, no database connections, no proprietary serialization. This follows Kernighan and Pike's argument that text is the universal interface (kernighan1984,). Any tool that can read a text file can participate in the workflow. Any human who can open a text editor can inspect or modify any artifact.

Layered context loading. Agents load only the context they need for the current stage, following the principle that less irrelevant context means better model performance (liu2024,). This is prevention rather than compression (jiang2023,). Within the content layers, ICM further distinguishes between reference material (stable rules and conventions that persist across runs) and working artifacts (per-run content that changes every time). The model receives these as structurally separate context, which matters because they require different kinds of attention: reference material should be internalized as constraints, while working artifacts should be processed as input.

Every output is an edit surface. The intermediate output of each stage is a file a human can open, read, edit, and save before the next stage runs. This implements Horvitz's mixed-initiative principles (horvitz1999,) and Shneiderman's direct manipulation paradigm (shneiderman1983,): the human works with visible, manipulable objects, and the system picks up whatever the human left there.

Configure the factory, not the product. A workspace is set up once with the user's preferences, brand, style, and structural decisions. After that, each run of the pipeline produces a new deliverable using the same configuration. This follows the continuous delivery principle that production pipelines should be repeatable (humble2010,).

3.2. Architecture

An ICM workspace is a folder. Inside it, agents navigate a five-layer context hierarchy (Figure 1).

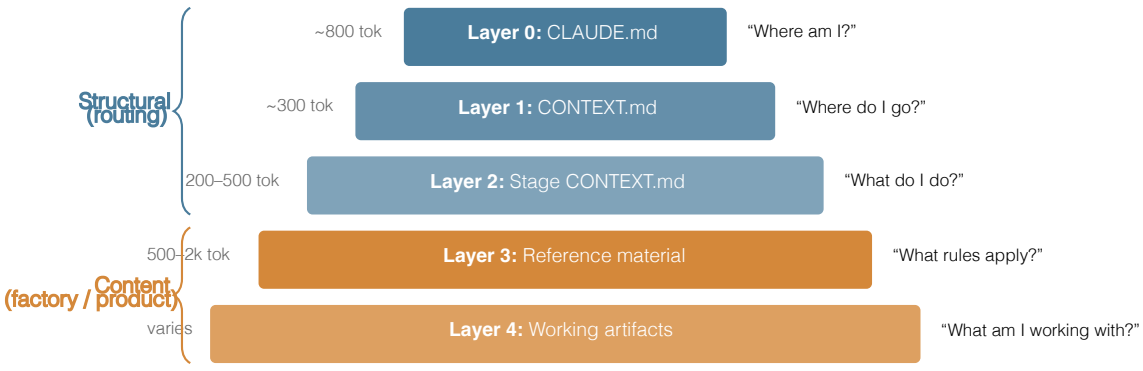

Figure 1. The five-layer context hierarchy. Layers 0–2 provide structural routing and stage instructions. Layers 3 and 4 carry content: Layer 3 holds reference material (the factory), stable across runs; Layer 4 holds working artifacts (the product), unique to each run.

Layer 0 is the global identity file. It tells the agent which workspace it is in, what the folder structure contains, and where to find things. Layer 1 is workspace-level task routing: given what the user wants to do, which stage handles it, and what shared resources exist across stages. Layer 2 is stage-specific: the contract that defines inputs, process, and outputs for one step of the workflow.

Layers 3 and 4 are both content that the agent loads while executing a stage, but they represent fundamentally different kinds of context.

Layer 3 is reference material: design systems, voice rules, build conventions, style guides, domain knowledge bundled as skill files. These files are configured once during workspace setup and remain stable across every run of the pipeline. They are the factory.[3]

[3]This connects to the fifth design principle: configure the factory, not the product. Layer 3 is the factory configuration. Layer 4 is what the factory produces each time it runs.

Layer 4 is working artifacts: the output of the previous stage, user-provided source material, anything specific to this particular run of the pipeline. These files are produced and consumed during execution and change every time.

The distinction matters for how the model processes context. Layer 3 material needs to be internalized as constraints and patterns: the model should write *like this*, use *these colors*, follow *these conventions*. Layer 4 material needs to be processed as input: the model should transform *this research* into a script, or convert *this script* into a visual specification. Mixing persistent rules with per-run artifacts in an undifferentiated context window forces the model to sort them on its own. Separating them in the folder structure means the model receives already-organized context.

Table 2. Layer 3 (reference material) versus Layer 4 (working artifacts).

	Layer 3: Reference	**Layer 4: Working**
Changes between runs	No	Yes
Example files	voice.md, design-system.md, conventions.md	research-output.md, script-draft.md
Model should	Internalize as constraints	Process as input
Configured during	Workspace setup (once)	Pipeline execution (each run)

Folder location	`references/`, `_config/`, `shared/`	`output/`
Analogy	The recipe	The ingredients

A rendering agent might only need Layers 0 through 2. A script-writing agent reads down to Layer 4 to access both the voice rules (Layer 3) and the source material (Layer 4). No agent reads everything. This keeps token cost low and context focused, and it avoids the degradation that Liu et al. documented when models process long contexts full of irrelevant material (liu2024,).

The folder structure for a typical workspace is shown in Figure 2.

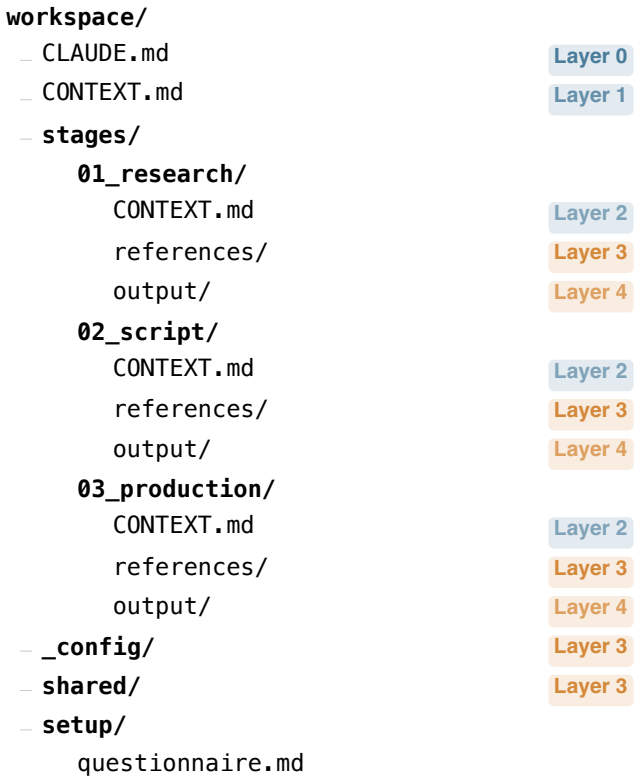

Figure 2. Folder structure of a typical ICM workspace, with layer annotations. Files and folders are color-coded by their role in the context hierarchy. Layer 3 material (reference) persists across runs. Layer 4 material (working artifacts) changes each time the pipeline executes.

The numbering encodes execution order. The folder boundaries enforce separation of concerns. The `output/` directories are the Layer 4 handoff points: the output of stage 01 becomes available as input to stage 02. If a human edits a file in `01_research/output/` before running stage 02, the agent picks up the edited version. The `references/` directories and `_config/` folder hold Layer 3 material: the stable knowledge and constraints that persist across runs.

Layer 2 is the control point of the entire system. Each stage contract includes an Inputs table that specifies exactly which files from Layers 3 and 4 the agent should load, and which sections of those files are relevant.[4]

[4]Larger reference collections can include their own routing files, a CONTEXT.md within a configuration or design system folder, that help agents navigate to the right content within the collection. This is the routing pattern from Layer 1 applied recursively within Layer 3.

Without this scoping mechanism, an agent would either load everything in the workspace or rely on its own judgment about what matters. The Inputs table makes the selection explicit, editable, and auditable.

This is the filesystem doing the work that a framework would otherwise do in code. Stage sequencing is the folder numbering. Context scoping is the folder hierarchy. State management is the files on disk. Coordination between stages is one folder's output being another folder's input.

From the model's perspective, this layered loading changes the composition of the context window at each stage. Layers 0 through 2 together contribute roughly 1,300 to 1,600 tokens of identity, routing, and stage-specific instruction. Layer 3 adds reference material scoped to the current stage, typically 500 to 2,000 tokens depending on how many conventions and guidelines apply. Layer 4 adds the working material for this run, a research document, a script, a specification, which varies with the content but rarely exceeds a few thousand tokens when the previous stage has done its job of condensing and structuring. The total context delivered to the model at any given stage typically ranges from 2,000 to 8,000 tokens, well within the range where current models perform at their best. Figure 3 illustrates this composition across three example stages and contrasts it with a monolithic approach.

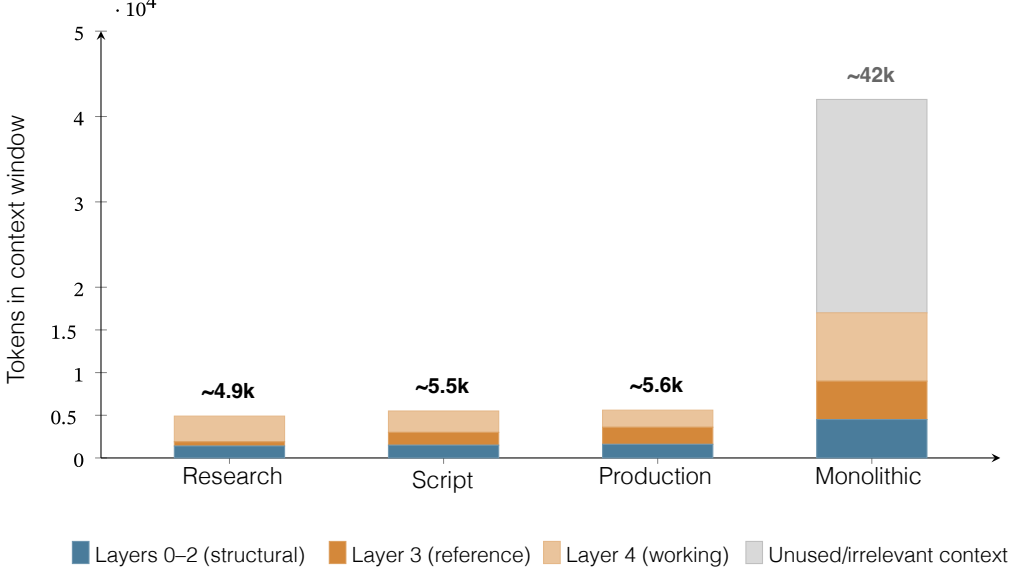

Figure 3. Context window composition by stage (representative token counts from the script-to-animation workspace). The three ICM stages each deliver 2,000–8,000 focused tokens. A monolithic approach loading all stages' instructions, all reference material, and all prior outputs produces a context window exceeding 40,000 tokens, most of it irrelevant to the current task.

Contrast this with a monolithic approach where all stage instructions, all reference files, and all prior outputs are loaded into a single prompt. That approach can easily reach 30,000 to 50,000 tokens, pushing into the range where Liu et al. found significant performance degradation on information retrieval tasks (liu2024,). The "unused/irrelevant context" segment in the monolithic bar of Figure 3 represents tokens from stages other than the one currently executing: instructions the agent will not follow during this step, reference material that applies to a different stage, and prior outputs already consumed by earlier stages. In ICM, these tokens are never loaded. In a monolithic prompt, they occupy context space without contributing to the current task. The compression research by Jiang et al. (jiang2023,; jiang2024,) addresses this problem after the fact. ICM's architecture avoids it by construction: each stage receives a focused, appropriately sized context window because the folder structure determines what gets loaded.

Richard Gabriel argued that systems prioritizing simplicity of implementation over feature completeness tend to survive and spread, because they are easier to port, easier to understand, and easier to improve incrementally (gabriel1991,). ICM trades the flexibility of a programmatic orchestrator for the portability, inspectability, and editability of plain files. That tradeoff is the point.

In the same spirit, Plan 9 from Bell Labs extended Unix's "everything is a file" principle to its full conclusion, representing all system resources as files in per-process namespaces (pike1995,). ICM applies the same idea to AI workflows: all state, all context, all instructions exist as files in a folder namespace.

3.3. Stage Contracts and Handoffs

Figure 4 illustrates the flow between stages. Each stage reads from the previous stage's output folder, processes it according to its own contract, and writes to its own output folder. At each boundary, the human can inspect and edit the output before the next stage runs.

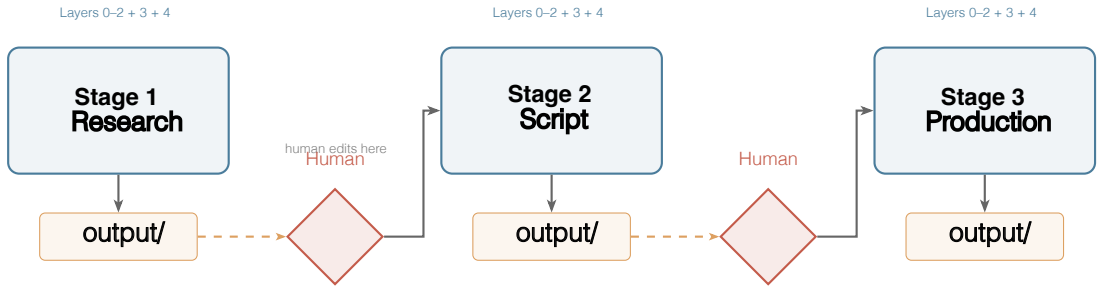

Figure 4. Pipeline flow through three stages with review gates. Each stage receives its own context (Layers 0–4), writes output to its folder, and the human reviews and optionally edits before the next stage reads it. The same model executes every stage; the folder structure controls what context it receives.

Each stage in an ICM workspace defines a contract with three parts: what it reads (inputs), what it does (process), and what it writes (outputs). This contract is spelled out in the stage's `CONTEXT.md` file.

A typical stage contract looks like this:

```
## Inputs
- Layer 4 (working): ../01_research/output/
- Layer 3 (reference): ../../_config/voice.md
- Layer 3 (reference): references/structure.md

## Process
Write a script based on the research output.
Follow the structure in structure.md.
Match the tone described in voice.md.

## Outputs
- script_draft.md -> output/
```

The Inputs table distinguishes between Layer 3 files (reference material that stays the same every run) and Layer 4 files (working artifacts from this specific run). The agent reads the `CONTEXT.md`, follows the instructions, and writes its output. The human reviews what landed in `output/`. If it needs adjustment, the human edits the file directly. The next stage reads whatever is there.

This implements prompt chaining at the filesystem level. Wu, Terry, and Cai introduced AI Chains as a method for creating transparent, controllable multi-step LLM workflows where each step's output becomes the next step's input (wu2022chains,). ICM does the same thing, but the chain is a sequence of folders and the links between them are plain files. The stage outputs serve as intermediate representations: each one is a complete, readable artifact that captures the work done so far and provides everything the next stage needs to continue.

There is also something of Knuth's literate programming in this design (knuth1984,). The markdown files that instruct the agent are simultaneously the documentation that tells a human what the stage does, what it expects, and what it produces. The instruction set and the documentation are the same artifact. This is useful in practice because it means the workspace is self-documenting. A new team member can read the `CONTEXT.md` files top to bottom and understand the entire pipeline without running it.

Wei et al. demonstrated that breaking complex reasoning into intermediate steps dramatically improves LLM performance (wei2022,). ICM applies this finding architecturally: complex workflows are decomposed into stages with explicit boundaries, and each stage receives focused, stage-appropriate context. The model gets a clear, scoped task at each step rather than a monolithic instruction to do everything in a single pass.

3.4. Portability and Reproducibility

A workspace is a folder. It can be copied to another machine, committed to Git, emailed as a zip file, or synced through any cloud storage service. It carries its own prompts, its own context structure, its own stage definitions. There is no server to configure, no environment to replicate, no deployment step.

ICM workspaces are Git-compatible by default (chacon2014,). Every change to a prompt, every edit to a stage output, every configuration adjustment is diffable and reversible. Stage outputs can be committed after each run, creating a version history of the entire production pipeline's behavior over time. This is infrastructure as code (morris2021,) applied to AI workflows: the workspace definition is the system. There is no separate deployment artifact.

This portability matters for a practical reason. If a consultant builds a workspace for a client's weekly reporting workflow, handing it over means copying a folder. The client can run it, edit the prompts to match their evolving needs, and adjust stages without involving a developer. The same handoff with a framework-based solution typically requires documentation, environment setup, dependency management, and ongoing technical support.

4. Working Implementations

ICM is not a theoretical proposal. The protocol has been implemented and tested across several production workflows.[5]

[5]All workspaces referenced here are available or buildable through the ICM repository at https://github.com/RinDig/Interpretable-Context-Methodology-ICM-.

4.1. Model and Environment

All workspaces described here were developed and run using Claude Code with Claude Opus 4.6 as the primary agent (anthropic2026opus,). For sub-agent tasks within stages, Opus 4.6 delegates to Claude Sonnet 4.6 through its Agent Teams capability, which coordinates multiple agents working in parallel from a single orchestrator.

A detail worth noting: Opus 4.6 uses the workspace's own context files, the `CONTEXT.md` hierarchy and Layer 3 reference material, to fill prompts for its sub-agents. The model reads the folder structure to determine what context each sub-agent should receive and what task it should perform. This means the ICM architecture is doing double duty. It structures context for the primary agent, and it provides the specification that the primary agent uses to delegate work. The folder hierarchy is both the human's control surface and the model's orchestration logic.

ICM is designed to be model-agnostic. The protocol specifies folder structure, file formats, and naming conventions. It does not depend on any model-specific capability. A workspace built for Claude could be run with a different model by pointing that model at the same files. Whether the results would be equivalent is an empirical question that depends on how different models handle the same context, but the protocol itself imposes no vendor lock-in. The workspaces described below were tested with the models listed above.

4.2. Script-to-Animation Pipeline

The first workspace built on ICM takes a content idea through three stages to produce a working animated video.

Stage 1 (`01_research`) takes a topic and produces structured research output: key points, narrative angles, supporting data. The agent reads a research brief from the user and writes a research document to its output folder.

Stage 2 (`02_script`) reads the research output and writes a script. The stage's `CONTEXT.md` points the agent to a voice guide and structural template in the `_config/` folder. The script follows the user's established tone and format.

Stage 3 (`03_production`) reads the finished script and produces animation specifications and working Remotion[6]

[6]Remotion is a React-based framework for creating videos programmatically.

code. The stage's context includes design guidelines, color palettes, and animation conventions from setup.

At each stage boundary, the human reviews the output. A research document that misses an important angle gets edited before the script stage runs. A script that runs too long gets trimmed before the production stage sees it. The agent at each stage works with whatever the human left in the previous output folder.

This workspace runs on a single Claude Code session. One orchestrating agent (Opus 4.6) manages the pipeline, delegating sub-tasks within stages to faster sub-agents (Sonnet 4.6) as described in Section 4.1. The delegation is itself driven by the folder structure: the orchestrating agent reads the stage's `CONTEXT.md` to determine what work to delegate and what context to provide. There is no separate orchestration framework. The same folder hierarchy that tells the human what each stage does tells the agent how to coordinate its sub-agents. In compiler terms, the workspace performs multi-pass compilation: the processing engine runs multiple times, producing a different intermediate representation at each pass, with the folder structure determining which pass runs next.

4.3. Course Deck Production

A second workspace takes unstructured source material (PDFs, papers, lecture notes, rough outlines) and produces polished PowerPoint slide decks through five stages: content extraction, structural planning, slide drafting, visual design specification, and final assembly.

The five-stage structure matters because slide deck production is a process where human judgment is essential at several points. The structural plan (stage 2 output) determines the entire arc of the presentation. Getting it wrong means everything downstream is wrong. By surfacing the structural plan as an editable markdown file before any slides are drafted, ICM lets the human course-correct at the point where correction is cheapest and most effective.

4.4. Building New Workspaces

ICM includes a workspace-builder: a five-stage workspace whose output is a new workspace. It walks through discovery (what is the domain, what is the workflow), stage mapping (where are the natural breakpoints), scaffolding (creating the folder structure), questionnaire design (what setup questions should the workspace ask), and validation (does the pipeline run end to end).

The workspace-builder itself follows ICM conventions. The workspaces it produces are consistent because the builder enforces the same structural rules it was built with.

This means practitioners can create new workspaces for their own domains without understanding the underlying conventions in detail. The builder encodes the conventions into its process. A marketing team can build a workspace for campaign production. A research group can build one for literature review and synthesis. A consultancy can build one for client deliverable pipelines. Each workspace is a folder they own and control.

ICM workspaces have been adopted by groups outside the author's organization. Researchers at the University of Edinburgh's Neuropolitics Lab have built workspaces for their domain, and teams at ICR Research and the Academy of International Affairs in Bonn are developing workspaces for their own workflows. The details of these implementations are limited by nondisclosure agreements, but their existence is noted here because the reviewer's natural question, does ICM work when someone other than its designer builds and operates the workspace, has at least a preliminary answer: yes, across academic research, policy analysis, and content production. A structured study of these external deployments is a clear next step.

4.5. Early Practitioner Experience

ICM has been used in production across content creation, training material development, research analysis, and policy workflows. The observations reported here are drawn from an invite-only practitioner community of 52 members whose backgrounds range from AI engineers and software developers to business owners, content creators, and academic researchers. These observations come from ongoing conversations with community members rather than from formal data collection protocols. They should be read as practitioner reports rather than controlled findings, but they reflect a broader base of experience than the author's own use alone.

The most consistent observation is where people choose to intervene (Figure 5). Across 33 community members who have used the script-to-animation workspace or structurally similar multi-stage workspaces, 30 report an intervention pattern consistent with a U-shape: heavy editing at stage 1 (direction-setting), light editing at the middle stages, and heavy editing again at the final stage (aligning output with earlier decisions). The remaining three report roughly equal editing across all stages. These numbers come from practitioner conversations, not from instrumented measurement, and should be interpreted accordingly.

The two peaks reflect different kinds of editing. Stage 1 editing is directional: the user is narrowing from broad possibilities to a specific angle, deciding what the piece is about. This is creative judgment. Final-stage editing is alignment work: the user is checking that the output faithfully represents decisions made in earlier stages. This is closer to debugging. The practitioner traces a misalignment in the output back through the pipeline to find where it diverged from the source material. Section 6 explores what tooling for this kind of traceability might look like.

The middle stages get the lightest touch because they sit between well-defined anchors. The earlier stage output sets the direction. The reference material (Layer 3 voice guides, structural templates) constrains the execution. With both anchors in place, the middle stages have less room to go wrong, and practitioners tend to trust them. This aligns with Parasuraman, Sheridan, and Wickens's observation that appropriate automation levels vary by task function (parasuraman2000,).

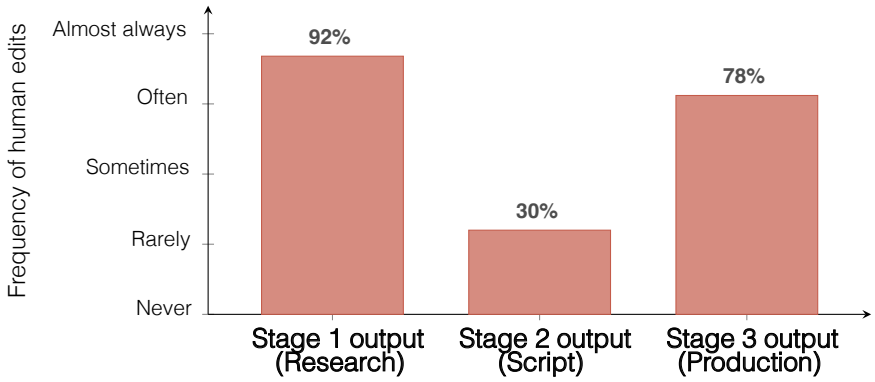

Figure 5. Observed frequency of human edits at each stage boundary, reported by 33 practitioners using multi-stage ICM workspaces. Intervention follows a U-shaped pattern: heavy at stage 1 (direction-setting), light at middle stages (constrained execution), heavy again at the final stage (aligning output with earlier decisions). Stage 1 editing is creative judgment. Final-stage editing is closer to debugging. Values are approximate and based on practitioner self-report through conversation, not instrumented measurement.

A second pattern involves prompt editing. Non-technical users, people without development experience, have successfully modified stage behavior by editing the markdown `CONTEXT.md` files. Changes include adjusting tone instructions, adding constraints ("keep scripts under 90 seconds"), and reordering the emphasis within a stage's process description. These edits would be equivalent to modifying agent configuration in a framework-based system, a task that typically requires a developer. The plain-text interface lowers this barrier in practice.

A third pattern is worth noting for its implications about accessibility. Three community members with no prior coding experience and no previous exposure to Claude Code used the ICM workspace-builder's questionnaire and setup process to create and run workspaces that produced ten-minute animated videos from scripts. They edited `CONTEXT.md` files, reviewed stage outputs, and iterated on their workspaces without developer assistance. This is a single data point from a small group, but it suggests that the filesystem interface can make AI agent orchestration accessible to people who would not be able to use a framework-based system at all.

A fourth pattern is workspace duplication. Users who have a working workspace for one content format (say, short explainer videos) duplicate the folder, modify the stage prompts to target a different format (say, long-form essays), and run the new workspace without rebuilding from scratch. The

workspace-builder supports creating workspaces from nothing, but in practice people often prefer to copy and adapt an existing one. This mirrors how Unix users build new shell scripts by modifying existing ones rather than starting from a blank file.

These observations are drawn from a community of varied backgrounds across a growing but still limited set of workflow types. A structured evaluation with formal data collection, systematic interviews, and controlled comparisons would be needed to draw firm conclusions about the generality of these patterns. The observations are reported here because they informed the protocol's design evolution and because they suggest directions for future study.

4.6. Threats to Validity

Several limitations constrain the conclusions that can be drawn from the current work, and naming them is important for interpreting the results above. The practitioner community provides a broader evidence base than single-author observation, but data collection has been informal: observations come from ongoing conversations rather than structured interviews, diary studies, or instrumented usage logging. The community is invite-only and self-selected, introducing both selection bias and potential enthusiasm bias. The reported intervention patterns (30 of 33 practitioners observing a U-shape) are self-reported through conversation and have not been verified through controlled measurement.

While ICM has been adopted across content production, academic research, and policy analysis workflows, the majority of active use remains concentrated in content production. The academic and policy deployments are early-stage, and their outcomes cannot yet be reported in detail. All testing was conducted using a single model family (Claude Opus 4.6 and Sonnet 4.6). Cross-model evaluation is a natural next step but falls outside the scope of this paper, which focuses on the architectural pattern and its interaction properties rather than model-specific performance. Output quality may vary with other models, particularly those with different context-handling characteristics.

No controlled comparison has been conducted between ICM's staged context loading and a monolithic prompting approach on the same tasks, so the claim that scoped context improves output quality rests on the theoretical support from the "lost in the middle" literature (liu2024,) and practitioner judgment rather than measured effect sizes. A formal user study with systematic data collection, structured interviews, and controlled comparisons across varied workflow types and participant backgrounds would substantially strengthen the empirical foundations of this work.

5. Discussion

5.1. Where This Works

ICM handles sequential multi-step workflows where a human reviews output at each stage. In practice, the protocol has been applied to content production pipelines (script-to-animation, short-form video), training material development (slide deck generation from source material), academic re-

search workflows (at the University of Edinburgh and ICR Research), and policy analysis (at the Academy of International Affairs, Bonn). The common thread across these deployments is that the workflows are sequential, the outputs benefit from human review at each step, and the same pipeline runs repeatedly with different input.

The common thread is that these workflows are sequential (step 2 follows step 1), reviewable (a human should check each step's output), and repeatable (the same pipeline runs weekly or daily with different input). For this class of workflow, ICM provides full orchestration capability with no framework code, no server infrastructure, and no developer dependency for day-to-day operation.

5.2. Where This Does Not Work

ICM is not a replacement for multi-agent frameworks in every context.

Real-time multi-agent collaboration, where agents need to communicate dynamically and respond to each other's outputs in tight loops, requires the kind of message-passing infrastructure that AutoGen (wu2023autogen,) and similar frameworks provide. ICM's sequential, file-based handoffs are too slow for this.

High-concurrency systems where many users hit the same pipeline simultaneously need proper queueing, state isolation, and deployment infrastructure. ICM is local-first by design. Scaling it to concurrent users would require building the infrastructure ICM was designed to avoid.

Workflows that require complex branching logic based on AI decisions mid-pipeline are awkward in ICM. A human can make branching decisions between stages (run stage 3a instead of 3b based on what they see in the stage 2 output), but automated branching would require scripting that moves ICM toward being a framework itself.

These boundaries matter. The claim is not that ICM replaces existing tools across the board. The claim is that for a large and common class of workflows, the existing tools provide more complexity than the problem requires, and that complexity has real costs: opacity, fragility, developer dependency, and overhead that slows iteration.

5.3. Observability as a Side Effect

The most useful property of ICM may be one that was not designed as a feature. Because every intermediate output is a plain file, the system is observable by default. There is no logging layer to build, no dashboard to configure, no special tooling to inspect pipeline state. You open a folder and read the files.

Rudin argued that inherently interpretable systems should be preferred over post-hoc explanations of opaque ones (rudin2019,). ICM is a glass-box AI workflow. It did not become transparent through the addition of an explanation layer. It was never opaque in the first place, because every artifact is a plain-text file that a human can read.

Amershi et al.'s guidelines for human-AI interaction include "make clear what the system can do," "support efficient correction," and "support efficient dismissal" (amershi2019,). Stage contracts make capabilities explicit. Markdown files support efficient correction (open, edit, save). Review gates at every stage boundary support dismissal (decide not to proceed, re-run the previous stage with different input, or abandon the run entirely).

The regulatory landscape may also be relevant here. The EU AI Act's human oversight requirements (enqvist2023,; novelli2024,) emphasize staged review, audit trails, and defined intervention points. ICM produces these as a byproduct of its architecture: there is no way to run an ICM pipeline without generating inspectable intermediate artifacts, because the intermediate artifacts are how the stages communicate. Whether this constitutes compliance with specific regulatory requirements is a legal question this paper does not attempt to answer, but the structural alignment is worth noting.

5.4. Implications for Intelligent System Design

The discussion so far has focused on how ICM structures the human side of human-AI interaction: edit surfaces, review gates, observability. But the architecture also has implications for how the intelligent system itself performs, and these are worth examining.

The core mechanism is context scoping. By delivering different context to the same model at each stage, ICM changes the task the model is performing. A model that receives research instructions, source material, and a topic brief behaves differently from the same model receiving a script template, a voice guide, and a research summary. The model's capabilities do not change between stages. What changes is the information it has available when generating output. This is context engineering in practice: the performance of the system depends on what context is delivered, in what structure, and at what moment.

The Layer 3/Layer 4 distinction adds a further dimension. Reference material (Layer 3) and working artifacts (Layer 4) ask different things of the model. Reference material says: here are the rules, follow them. Working artifacts say: here is the input, transform it. Delivering these as structurally separate context, rather than mixing them in a single undifferentiated prompt, gives the model clearer signals about which information constrains its behavior and which information it should act on. Whether this structural separation measurably improves output quality compared to a flat context of equivalent content is an open empirical question, but early practitioner experience suggests that stages where reference and working material are clearly separated produce more consistent adherence to style and format guidelines.

This raises a question about the relationship between context structure and output quality. In early use, a pattern emerged: stages with tightly scoped context (clear instructions, limited reference material, a specific output format) produced more consistent results than stages with broad context (open-ended instructions, large volumes of reference material, loosely defined output expectations). This is consistent with the "lost in the middle" findings (liu2024,) and with the chain-of-thought literature showing that decomposed tasks outperform monolithic ones (wei2022,), but it suggests something more specific. The structure of the context delivery, how information is organized and bounded, may matter as much as the content of the context itself. ICM's folder-based scoping enforces this structure by default: each stage folder contains only what that stage needs, and the boundaries are visible and editable.

There are open questions here that the current work does not answer. First, does the five-layer hierarchy (workspace identity, task routing, stage contracts, reference material, working artifacts) generalize across model families, or is it tuned to the specific attention patterns of the models tested? The protocol is designed to be model-agnostic (Section 4.1), but all current testing has been conducted on a single model family. Cross-model evaluation, running the same workspace on Claude, GPT, Gemini, and open-weight models such as Llama, is a clear next step. This paper scopes that question as future work because the present contribution is the architectural pattern and its interaction properties, not a model-specific performance claim. Second, as context windows grow larger, does selective loading become less important? If a model can reliably attend to 200,000 tokens without degradation, the engineering argument for ICM's scoping weakens, though the human-interaction arguments (observability, editability, review gates) remain. Third, how sensitive is stage output quality to the ordering and formatting of context within a layer? The current protocol specifies what files a stage should load but does not prescribe the order in which they appear in the context window. Whether ordering matters at the scale of ICM's typical context sizes (2,000 to 8,000 tokens per stage) is an empirical question worth investigating.

These questions point toward a research program that sits at the intersection of context engineering and interaction design: understanding how the structure of information delivery to language models affects both the model's output quality and the human's ability to steer, inspect, and correct that output. ICM provides a concrete platform for investigating these questions because its architecture makes the context structure explicit, editable, and observable at every stage.

6. Future Directions: Compilation, Debugging, and Source Integrity

The previous sections describe ICM as it currently works in production. This section describes where it should go next. The ideas here are informed by early practitioner experience and by a structural analogy that the paper has not yet drawn: the relationship between ICM workspaces and multi-pass compilers.

6.1. ICM as Multi-Pass Incremental Compilation

The paper has grounded ICM in Unix pipelines, Make, and the pipe-and-filter pattern. There is a closer analogy that deserves attention: multi-pass compilation (aho2006,).

A multi-pass compiler transforms source code through a sequence of discrete passes. The lexer produces tokens. The parser produces a syntax tree. Semantic analysis annotates the tree. Optimization passes rewrite it. Code generation produces the final output. Each pass reads the output of the previous pass, transforms it according to its own rules, and writes an intermediate representation that the next pass can consume. The intermediate representations are well-defined, inspectable, and (in debugging builds) preserved for examination.

ICM does the same thing with content. Stage 1 (research) transforms a topic brief into structured research output. Stage 2 (script) transforms the research output into a script. Stage 3 (production) transforms the script into animation specifications and code. Each stage reads the previous stage's output, applies its own context and instructions, and writes an intermediate artifact that the next stage consumes. The intermediate artifacts are plain files that can be opened, read, and edited.

The analogy extends further. Incremental compilation means recompiling only the parts of the program that changed, rather than rebuilding from scratch. ICM supports this by default: if the research output is fine but the script needs rework, the practitioner re-runs stage 2 without touching stage 1. If a voice guide in the reference material changes, only the stages that load that file need to run again. The folder structure tracks these dependencies implicitly: a stage's Inputs table declares which files it reads, and a change to any of those files signals that the stage's output may be stale.

This is worth naming because it connects ICM to a body of compiler engineering that has spent fifty years solving the problems of pass decomposition, intermediate representation design, and selective recompilation. The current paper draws from Unix and software architecture. Future work should draw from compiler theory as well, particularly around dependency tracking, change propagation, and the formal properties of intermediate representations.

6.2. Toward Semantic Debugging

Traditional debugging rests on a simple principle: when the output is wrong, trace the failure back through the program's execution to find the instruction that caused it (zeller2009,). Debuggers provide tools for this: breakpoints that pause execution at specific instructions, stack traces that show the call chain, variable inspection that shows state at each point, and source maps that connect compiled output back to the original source.

ICM currently provides observability but not traceability. A practitioner can open any stage's output folder and read what the agent produced. But if a phrase in the stage 3 output sounds wrong, there is no direct way to trace that phrase back to the specific instruction, reference file, or previous stage output that caused it. The practitioner has to do this manually: read the stage 3 contract, check which files it loaded, read those files, and form a judgment about which source is responsible. This

works. It is how most ICM debugging happens today. But it is the equivalent of debugging a program by reading the source code and thinking hard, without a debugger.

The question is what a debugger for semantic content would look like. Several directions are worth exploring.

Output provenance through identifiers. If each section of a stage's output carried an identifier linking it to the source instruction or reference file that produced it, a practitioner could trace backward from any part of the output to the context that generated it. In compiler terms, this is the equivalent of debug symbols or source maps: metadata that connects output back to source without changing the output itself. In practice, this could mean embedding lightweight markers (GUIDs, section tags, or comment annotations) in stage output files that reference specific sections of the stage's `CONTEXT.md` or Layer 3 reference files.

Cross-stage trace verification. In the script-to-animation workspace described in Section 4, one recurring problem has been misalignment between the animation specification (stage 3 output) and the script (stage 2 output). Timing drifts. Animations reference phrases that were revised. Visual density does not match pacing. This is the source of the U-shaped intervention pattern observed in Section 4.4: the stage 3 editing that brings intervention back up is almost entirely alignment work, tracing the final output back through the pipeline to find where it diverged from earlier decisions. The current solution is an audit file that forces the agent to trace back from the specification to the original script, re-verifying timing for each phrase and flagging inconsistencies. This works well enough that it catches most alignment errors, and the errors it catches are remarkably consistent in kind: frame count discrepancies, visual density mismatches, and pacing breaks at scene boundaries.

This audit file is a proto-debugger. It implements a specific kind of cross-stage verification: checking that the output of stage n is consistent with the output of stage $n-2$ by re-reading both and comparing them against defined criteria. The pattern could be generalized. A stage contract could include a `Verify` section alongside its `Inputs`, `Process`, and `Outputs` sections, specifying which earlier stage outputs should be checked for consistency and what criteria to check against. The agent would run these verification checks as part of the stage's execution and flag discrepancies before the human reviews.

Breakpoints in markdown. The most speculative direction involves something like breakpoints for markdown files. In a traditional debugger, a breakpoint says "pause here and let me inspect the state." In ICM, a breakpoint in a `CONTEXT.md` file might say "after the agent processes this instruction, show me what it produced before continuing." This would be particularly useful in stages with complex instructions where the practitioner wants to verify that the agent interpreted a specific constraint correctly before it finishes the rest of the stage's work. It turns a single-pass stage into a sequence of verifiable sub-steps.

These ideas are not yet implemented. They are described here because the gap they address, the ability to trace output back to source, is a gap that compiler engineering solved decades ago and that ICM will need to solve as workspaces grow more complex.

6.3. Source Integrity and the Edit-Source Principle

The current paper describes ICM's review gates as places where practitioners edit stage output. This is useful and it works. But there is an argument, drawn directly from software engineering practice, that the source files should be what improves over time, and that editing output is treating symptoms rather than causes.

The argument is straightforward. If a script sounds wrong at stage 2, there are two possible responses. The first is to edit the script directly: fix the tone, adjust the phrasing, move on. The second is to ask why the script sounds wrong and trace the problem back to the source that produced it. Maybe the voice guide in the reference material is underspecified. Maybe the stage contract's instructions emphasize the wrong quality. Maybe the research output from stage 1 framed the topic in a way that led the script in the wrong direction. Editing the output fixes this run. Editing the source fixes every future run.

In compiler terms, editing the output is patching the binary. It works, but it does not improve the compiler. A developer who finds a bug in compiled code traces it back to the source and fixes it there, so that every subsequent build is correct.

For ICM, the tension is real. Creative content is fuzzier than compiled code. Sometimes the output needs a human touch that cannot be reduced to a source-level rule. A script might benefit from a turn of phrase that no amount of voice guide refinement would have produced. Editing the output in that case is the right move. The practitioner is adding value that the system cannot generate on its own.

But there is a class of output edits that are diagnostic. If the practitioner consistently tightens the opening paragraph, that is a signal that the stage contract should say "keep the opening under three sentences." If the tone drifts formal every time, that is a signal that the voice guide needs a stronger example of the target register. These recurring edits are debugging information. They point to fixable source-level problems.

A future version of ICM could support this by tracking output edits across runs. If a practitioner edits the same kind of thing in the same stage's output three runs in a row, the system could surface that pattern and suggest a source-level change: a contract amendment, a reference file update, a new constraint. This would close the loop between output editing and source improvement, turning one-off fixes into durable system improvements.

The principle matters because it addresses a question about ICM's long-term trajectory. If workspaces are only as good as the last human edit of their output, they remain tools. If workspaces improve their own source files over time, incorporating the patterns they learn from human corrections, they become systems that get better with use. The debugging and traceability infrastructure described in the previous subsection is a prerequisite for this: you cannot improve the source if you cannot trace the problem back to it.

7. Conclusion

The principles that made Unix pipelines effective in the 1970s apply to AI agent orchestration in the 2020s. Programs that do one thing. Output of one becomes input of another. Plain text as universal interface. Human-readable intermediate state.

ICM applies these principles to a specific problem: structuring context for AI agents across multi-step workflows. The result is a system where the folder structure replaces the framework. One agent reads different context at each stage rather than multiple agents coordinating through code. Local scripts handle the mechanical work that does not need AI. Every intermediate output is a file a human can read and edit.

For practitioners whose AI workflows are sequential, reviewable, and repeatable, this means full pipeline capability with no framework to learn, no server to maintain, and no developer needed for day-to-day operation. The workspace is a folder. It can be copied, versioned, shared, and edited with a text editor. The simplest viable architecture for this class of problem is one that already exists on every computer: the filesystem.

The protocol is open source under the MIT license and includes a workspace-builder for creating new workspaces across any domain.

References

(1) M. D. McIlroy, E. N. Pinson, and B. A. Tague, "Unix Time-Sharing System: Foreword," *The Bell System Technical Journal*, vol. 57, no. 6, part 2, pp. 1902–1903, 1978.

(2) D. M. Ritchie and K. Thompson, "The UNIX Time-Sharing System," *Communications of the ACM*, vol. 17, no. 7, pp. 365–375, 1974. DOI: https://doi.org/10.1145/361011.361061

(3) P. H. Salus, *A Quarter Century of Unix*. Addison-Wesley, 1994. ISBN: 0-201-54777-5.

(4) E. S. Raymond, *The Art of Unix Programming*. Addison-Wesley Professional, 2003. ISBN: 0-13-142901-9.
Available: http://www.catb.org/esr/writings/taoup/html/

(5) B. W. Kernighan and R. Pike, *The UNIX Programming Environment*. Prentice Hall, 1984. ISBN: 0-13-937681-X.

(6) M. Shaw and D. Garlan, *Software Architecture: Perspectives on an Emerging Discipline*. Prentice Hall, 1996. ISBN: 0-13-182957-2.

(7) S. I. Feldman, “Make — A Program for Maintaining Computer Programs,” *Software: Practice and Experience*, vol. 9, no. 4, pp. 255–265, 1979.
DOI: https://doi.org/10.1002/spe.4380090402

(8) E. W. Dijkstra, “On the Role of Scientific Thought,” Manuscript EWD447, 1974. Reprinted in *Selected Writings on Computing: A Personal Perspective*, pp. 60–66. Springer-Verlag, 1982.
Available: https://www.cs.utexas.edu/~EWD/transcriptions/EWD04xx/EWD447.html

(9) D. L. Parnas, “On the Criteria To Be Used in Decomposing Systems into Modules,” *Communications of the ACM*, vol. 15, no. 12, pp. 1053–1058, 1972.
DOI: https://doi.org/10.1145/361598.361623

(10) D. E. Knuth, “Literate Programming,” *The Computer Journal*, vol. 27, no. 2, pp. 97–111, 1984. DOI: https://doi.org/10.1093/comjnl/27.2.97

(11) R. P. Gabriel, “The Rise of ‘Worse is Better’,” Originally part of “Lisp: Good News, Bad News, How to Win Big.” *AI Expert*, vol. 6, no. 6, pp. 33–35, 1991. Available: https://www.dreamsongs.com/WorseIsBetter.html

(12) R. Pike, D. Presotto, S. Dorward, B. Flandrena, K. Thompson, H. Trickey, and P. Winterbottom, “Plan 9 from Bell Labs,” *Computing Systems*, vol. 8, no. 3, pp. 221–254, 1995. Available: https://css.csail.mit.edu/6.824/2014/papers/plan9.pdf

(13) S. Chacon and B. Straub, *Pro Git*, 2nd ed. Apress, 2014. ISBN: 978-1-4842-0076-6. Available: https://git-scm.com/book

(14) K. Morris, *Infrastructure as Code: Dynamic Systems for the Cloud Age*, 2nd ed. O’Reilly Media, 2021. ISBN: 978-1-098-11467-1.

(15) J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010. ISBN: 978-0-321-60191-9.

(16) A. Karpathy, “+1 for ‘context engineering’ over ‘prompt engineering’…,” X (formerly Twitter), June 25, 2025. Available: https://x.com/karpathy/status/1937902205765607626

(17) L. Martin, “Context Engineering,” LangChain Blog, July 2, 2025.
Available: https://blog.langchain.com/context-engineering-for-agents/

(18) S. Willison, “Context Engineering,” *Simon Willison’s Weblog*, June 27, 2025.
Available: https://simonwillison.net/2025/jun/27/context-engineering/

(19) DAIR.AI, “Context Engineering Guide,” *Prompting Guide*, 2025.
Available: https://www.promptingguide.ai/guides/context-engineering-guide

(20) Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, A. Awadallah, R. W. White, D. Burger, and C. Wang, “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation,” *COLM 2024*, arXiv:2308.08155, August 2023.
Available: https://arxiv.org/abs/2308.08155

(21) H. Chase, *LangChain* [open-source framework]. First released October 2022.
Available: https://github.com/langchain-ai/langchain

(22) J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative Agents: Interactive Simulacra of Human Behavior,” *Proceedings of UIST ’23*. ACM, 2023.
DOI: https://doi.org/10.1145/3586183.3606763

(23) Anthropic, “Introducing the Model Context Protocol,” Anthropic Blog, November 25, 2024.
Available: https://www.anthropic.com/news/model-context-protocol

(24) A. Jones and C. Kelly, “Code Execution with MCP,” Anthropic Engineering Blog, 2025. Available: https://www.anthropic.com/engineering/code-execution-with-mcp

(25) N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the Middle: How Language Models Use Long Contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024. Available: https://arxiv.org/abs/2307.03172

(26) T. Wu, M. Terry, and C. J. Cai, “AI Chains: Transparent and Controllable Human-AI Interaction by Chaining Large Language Model Prompts,” *CHI Conference on Human Factors in Computing Systems (CHI ’22)*. ACM, 2022. DOI: https://doi.org/10.1145/3491102.3517582

(27) J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” *NeurIPS 2022*. Available: https://arxiv.org/abs/2201.11903

(28) T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language Models Can Teach Themselves to Use Tools,” *NeurIPS 2023*. Available: https://arxiv.org/abs/2302.04761

(29) S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, “Gorilla: Large Language Model Connected with Massive APIs,” *NeurIPS 2024*, arXiv:2305.15334, 2023. Available: https://arxiv.org/abs/2305.15334

(30) P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS 2020*, pp. 9459–9474. Available: https://arxiv.org/abs/2005.11401

(31) H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, “LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models,” *EMNLP 2023*, pp. 13358–13376. DOI: https://doi.org/10.18653/v1/2023.emnlp-main.825

(32) H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, “LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression,” *ACL 2024*, pp. 1658–1677. Available: https://arxiv.org/abs/2310.06839

(33) Addyo, “Context Engineering: Bringing Engineering Discipline to Prompts,” Substack, 2025. Available: https://addyo.substack.com/p/context-engineering-bringing-engineering

(34) S. Amershi, M. Cakmak, W. B. Knox, and T. Kulesza, “Power to the People: The Role of Humans in Interactive Machine Learning,” *AI Magazine*, vol. 35, no. 4, pp. 105–120, 2014. DOI: https://doi.org/10.1609/aimag.v35i4.2513

(35) J. A. Fails and D. R. Olsen, Jr., “Interactive Machine Learning,” *Proceedings of IUI ’03*, pp. 39–45. ACM, 2003. DOI: https://doi.org/10.1145/604045.604056

(36) J. J. Dudley and P. O. Kristensson, “A Review of User Interface Design for Interactive Machine Learning,” *ACM Transactions on Interactive Intelligent Systems*, vol. 8, no. 2, Article 8, pp. 1–37, 2018. DOI: https://doi.org/10.1145/3185517

(37) E. Horvitz, “Principles of Mixed-Initiative User Interfaces,” *CHI ’99*, pp. 159–166. ACM, 1999. DOI: https://doi.org/10.1145/302979.303030

(38) M. T. Ribeiro, S. Singh, and C. Guestrin, “‘Why Should I Trust You?’: Explaining the Predictions of Any Classifier,” *KDD ’16*, pp. 1135–1144. ACM, 2016. DOI: https://doi.org/10.1145/2939672.2939778

(39) S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” *NeurIPS 2017*, pp. 4765–4774. Available: https://papers.nips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions

(40) J. D. Lee and K. A. See, “Trust in Automation: Designing for Appropriate Reliance,” *Human Factors*, vol. 46, no. 1, pp. 50–80, 2004. DOI: https://doi.org/10.1518/hfes.46.1.50_30392

(41) R. Parasuraman and V. Riley, “Humans and Automation: Use, Misuse, Disuse, Abuse,” *Human Factors*, vol. 39, no. 2, pp. 230–253, 1997. DOI: https://doi.org/10.1518/001872097778543886

(42) R. Parasuraman, T. B. Sheridan, and C. D. Wickens, “A Model for Types and Levels of Human Interaction with Automation,” *IEEE Transactions on Systems, Man, and Cybernetics — Part A*, vol. 30, no. 3, pp. 286–297, 2000.
DOI: https://doi.org/10.1109/3468.844354

(43) B. Shneiderman, “Human-Centered Artificial Intelligence: Reliable, Safe & Trustworthy,” *International Journal of Human–Computer Interaction*, vol. 36, no. 6, pp. 495–504, 2020.
DOI: https://doi.org/10.1080/10447318.2020.1741118

(44) B. Shneiderman, *Human-Centered AI*. Oxford University Press, 2022. ISBN: 978-0192845290.

(45) C. Rudin, “Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead,” *Nature Machine Intelligence*, vol. 1, pp. 206–215, 2019.
DOI: https://doi.org/10.1038/s42256-019-0048-x

(46) B. Shneiderman, “Direct Manipulation: A Step Beyond Programming Languages,” *IEEE Computer*, vol. 16, no. 8, pp. 57–69, 1983.
DOI: https://doi.org/10.1109/MC.1983.1654471

(47) S. Amershi, D. Weld, M. Vorvoreanu, A. Fourney, B. Nushi, P. Collisson, J. Suh, S. Iqbal, P. N. Bennett, K. Inkpen, J. Teevan, R. Kikin-Gil, and E. Horvitz, “Guidelines for Human-AI Interaction,” *CHI 2019*, Article 3, pp. 1–13. ACM, 2019.
DOI: https://doi.org/10.1145/3290605.3300233

(48) M. Zaharia, A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe, F. Xie, and C. Zumar, "Accelerating the Machine Learning Lifecycle with MLflow," *IEEE Data Engineering Bulletin*, vol. 41, no. 4, pp. 39–45, 2018. Available: https://people.eecs.berkeley.edu/~matei/papers/2018/ieee_mlflow.pdf

(49) L. Enqvist, "'Human Oversight' in the EU Artificial Intelligence Act," *The Theory and Practice of Legislation*, vol. 11, no. 3, 2023. DOI: https://doi.org/10.1080/17579961.2023.2245683

(50) C. Novelli, F. Casolari, A. Rotolo, M. Taddeo, and L. Floridi, "Institutionalised Distrust and Human Oversight of Artificial Intelligence," *Digital Society*, vol. 3, no. 8, 2024. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC11614927/

(51) M. Fink, "Human Oversight under Article 14 of the EU AI Act," SSRN: 5147196, 2025. Forthcoming in Malgieri et al. (eds.), *AI Act Commentary*. Hart-Bloomsbury, 2026. DOI: https://doi.org/10.2139/ssrn.5147196

(52) A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006. ISBN: 978-0-321-48681-3.

(53) A. Zeller, *Why Programs Fail: A Guide to Systematic Debugging*, 2nd ed. Morgan Kaufmann, 2009. ISBN: 978-0-12-374515-6.

(54) Anthropic, "Introducing Claude Opus 4.6," https://www.anthropic.com/news/claude-opus-4-6, February 2026.

Instructions for reporting errors

We are continuing to improve HTML versions of papers, and your feedback helps enhance accessibility and mobile support. To report errors in the HTML that will help us improve conversion and rendering, choose any of the methods listed below:

- Click the "Report Issue" (🐞) button, located in the page header.

Tip: You can select the relevant text first, to include it in your report.

Our team has already identified the following issues. We appreciate your time reviewing and reporting rendering errors we may not have found yet. Your efforts will help us improve the HTML versions for all readers, because disability should not be a barrier to accessing research. Thank you for your continued support in championing open access for all.

Have a free development cycle? Help support accessibility at arXiv! Our collaborators at LaTeXML maintain a list of packages that need conversion, and welcome developer contributions.